Name : Ashwin Rai Roll No. : 202403069 Class : TYCS – A

## Practical No. 2: RSA Encryption and Decryption

### Aim

To implement the RSA algorithm for public-key encryption and decryption and understand the concepts of asymmetric cryptography.

### Objectives

- To understand the working of public-key cryptography.
- To generate RSA public and private keys.
- To perform encryption using the public key.
- To perform decryption using the private key.
- To study the security features of the RSA algorithm.

### Theory

### 1. RSA Algorithm

RSA (Rivest–Shamir–Adleman) is one of the most widely used public-key cryptographic algorithms. Unlike classical ciphers, RSA uses **two different keys**:

- **Public Key** – Used for encryption and can be shared with anyone.
- **Private Key** – Used for decryption and kept secret by the owner.

RSA is based on the mathematical difficulty of factoring the product of two large prime numbers.

### Key Generation Steps

1. Select two prime numbers **p** and **q**.
2. Compute:
    - $\mathbf{n = p \times q}$
3. Compute Euler's Totient:
    - $\varphi(n) = (p - 1) \times (q - 1)$
4. Choose a public exponent **e** such that:
    - $\gcd(e, \varphi(n)) = 1$
5. Compute the private exponent **d** such that:
    - $\mathbf{d \times e \equiv 1 \pmod{\varphi(n)}}$

The generated keys are:

- **Public Key = (n, e)**
- **Private Key = (n, d)**

**RSA Encryption**

The sender encrypts the plaintext using the receiver's public key.

**Formula:**

$$\mathbf{C = P^e \bmod n}$$

Where:

- **P** = Plaintext
- **C** = Ciphertext
- **e** = Public exponent
- **n** = Product of two prime numbers

**RSA Decryption**

The receiver decrypts the ciphertext using the private key.

**Formula:**

$$\mathbf{P = C^d \bmod n}$$

Where:

- **C** = Ciphertext
- **P** = Plaintext
- **d** = Private exponent

**Algorithm**

**A. RSA Key Generation**

1. Generate two prime numbers **p** and **q**.
2. Compute **n = p × q**.
3. Compute $\varphi(n) = (p - 1)(q - 1)$.
4. Choose a public exponent **e**.
5. Compute the private exponent **d** using the modular inverse.

6. Generate the public and private keys.

**B. RSA Encryption**

1. Read the plaintext message.
2. Read the public key.
3. Convert each character into its ASCII value.
4. Encrypt each character using:
    - **Cipher = Plaintext$^{e}$ mod n**
5. Display the encrypted message.

**C. RSA Decryption**

1. Read the encrypted message.
2. Read the private key.
3. Decrypt each encrypted value using:
    - **Plaintext = Cipher$^{d}$ mod n**
4. Convert ASCII values back to characters.
5. Display the original plaintext.

**Program: RSA Encryption and Decryption**

```
import random
import math

def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

def generate_prime(bits):
    while True:
        num = random.getrandbits(bits)
        if is_prime(num):
            return num

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
```

```
    return a

def mod_inverse(a, m):
    m0, x0, x1 = m, 0, 1
    while a > 1:
        q = a // m
        m, a = a % m, m
        x0, x1 = x1 - q * x0, x0
    return x1 if x1 >= 0 else x1 + m0

def generate_keypair(bits):
    p = generate_prime(bits)
    q = generate_prime(bits)

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    d = mod_inverse(e, phi)

    return ((n, e), (n, d))

def encrypt(message, public_key):
    n, e = public_key
    return [pow(ord(char), e, n) for char in message]

def decrypt(ciphertext, private_key):
    n, d = private_key
    return ''.join(chr(pow(char, d, n)) for char in ciphertext)

if __name__ == "__main__":

    bits = 16

    public_key, private_key = generate_keypair(bits)

    print("Public Key (n, e):", public_key)
    print("Private Key (n, d):", private_key)

    plaintext = input("Enter the Plain Text: ")
```

```
ciphertext = encrypt(plaintext, public_key)
print("Encrypted Message:", ciphertext)

decrypted_text = decrypt(ciphertext, private_key)
print("Decrypted Message:", decrypted_text)
```

```
========== RESTART: C:/Users/CC2_50/Documents/CIS_Prac2.py ==========
Public Key (n, e): (343672489, 65537)
Private Key (n, d): (343672489, 168565121)
Enter the Plain Text: Ashwin
Encrypted Message: [124714054, 131576928, 65672845, 342572789, 177270864, 186079284]
Decrypted Message: Ashwin
```